Stat 184 R HW 4.3

Stat 184 R QMD assignment

Siddhant Bhagat

2025-04-14

Table of contents

1 World's Busiest Airports **1**
1.1 Passenger Traffic Summary Table . . . 2
1.2 Passenger Traffic Recovery Trends . . . 2

2 Monte Carlo Numerical Integration **4**
2.1 Small Multiple Visualization . . . 4
2.2 Interpretation . . . 5

3 GenAI Prompting: Plan-Informed Vs Generic Vs Self Made **6**
3.0.1 Goal . . . 6
3.0.2 Needs . . . 6
3.0.3 Plan . . . 7
3.1 Planned AI Response . . . 7
3.2 Self Made Response . . . 9
3.3 Generic Prompt Response (Anthropic Claude) . . . 13
3.4 *Comparison* . . . 14

4 Self-Reflection **16**

5 GenAI/LLM Usage Appendix **17**
5.1 Usage 1 — Plan-Informed Prompt . . . 17
5.2 Usage 2 — Generic Prompt . . . 20

1 World's Busiest Airports

The following code analyzes traffic data for six of the world's busiest airports from 2020 through 2024. The six airports selected are Hartsfield Jackson Atlanta International Airport, Frankfurt

Airport, Beijing Daxing International Airport, Dallas Fort Worth International Airport, Dubai International Airport, and the Indira Gandhi International Airport . These airports serve hundreds of millions of passengers annually, making them helpful for understanding global aviation trends.

1.1 Passenger Traffic Summary Table

The table below shows total annual passenger counts for each airport from 2020 to 2024, along with the absolute and percentage change over that period. Please ote that some airports have missing values for certain years, due to incomplete data availability on Wikipedia

Table 1: Annual Passenger Traffic for Six Major Airports (2020–2024)

Airport	2020	2021	2022	2023	2024	Change (2020–2024)	% Change
Beijing Daxing	—	25,051,012	—	—	49,441,029	—	—
Dallas Fort Worth	39,364,990	62,465,756	73,362,946	81,755,538	87,817,864	48,452,874	123.09%
Dubai	25,836,771	29,110,609	66,069,981	86,994,365	92,331,506	66,494,735	257.36%
Frankfurt	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	42,796,356	228.02%
Hartsfield–Jackson Atlanta	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	65,149,081	151.8%
Indira Gandhi	28,500,545	37,139,957	59,490,074	72,214,841	77,820,834	49,320,289	173.05%

1.2 Passenger Traffic Recovery Trends

The figure below plots each airport's percent change in annual passengers relative to its 2020 baseline. The year 2020 represents the nadir of global aviation activity due to the pandemic in 2020, so positive values in subsequent years signal a recovery trajectory (which the table also shows)

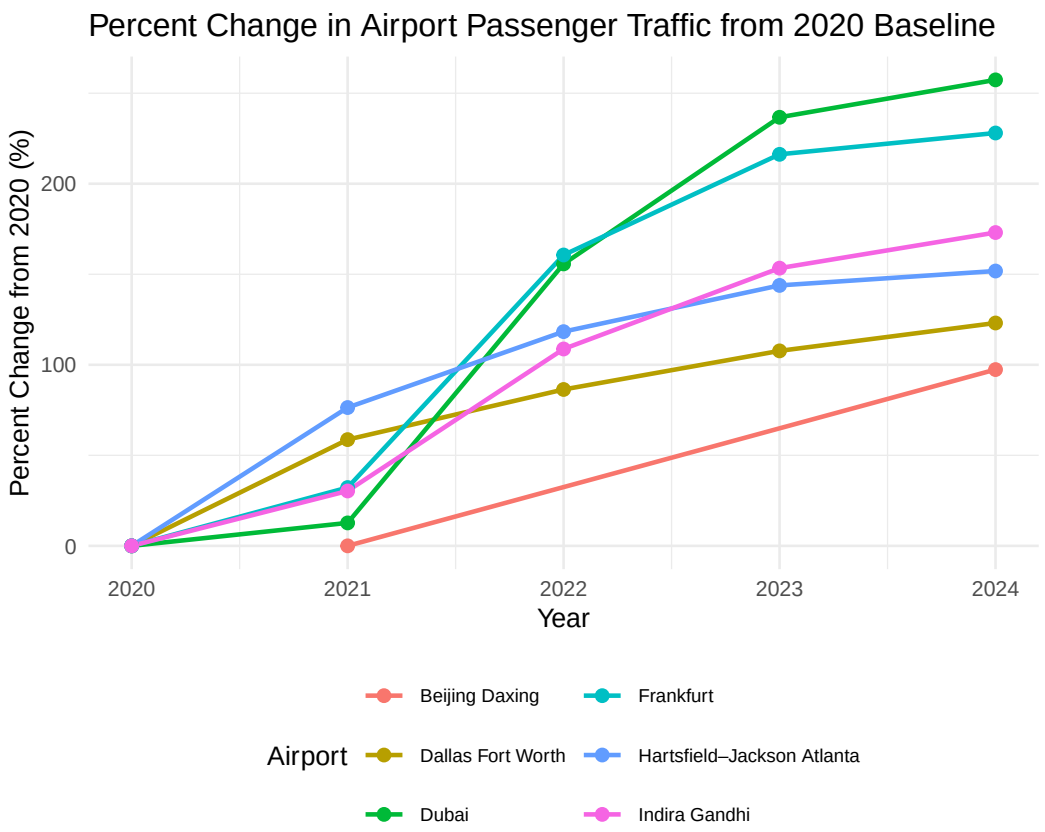


Figure 1: Percent change in annual passenger traffic relative to 2020 for six major airports.

The data reveal a dramatic rebound in passenger traffic across all six airports following the historic lows of 2020(which we can infer due to covid). Dubai International and Hartsfield Jackson Atlanta both show consistent growth, reflecting the outsized role of hub airports in global connectivity. Frankfurt and Dallas Fort Worth also demonstrate strong recoveries, while data gaps for Beijing Daxing limit direct comparison for some years. By 2024, most airports had surpassed or closely approached their pre pandemic passenger volumes (some even exceeding it).

---

2 Monte Carlo Numerical Integration

Monte Carlo simulation is a computational technique that uses random sampling to estimate quantities that may be difficult to compute analytically. In the context of numerical integration, random points are scattered uniformly across a bounding rectangle, and the proportion of points that fall at or below a target curve is used to estimate the area under that curve — and therefore the definite integral.

In this analysis, we apply Monte Carlo integration to estimate the integral of the standard normal probability density function over the interval $[-4, 4]$:

$$\int_{-4}^{4} \frac{1}{\sqrt{2\pi}} e^{-x^2/2}\, dx$$

Because the standard normal PDF integrates to 1 over the entire real line, and because almost all of its probability mass lies within ± 4 standard deviations, the exact answer is very close to **1.000** (specifically, approximately 0.99994).

2.1 Small Multiple Visualization

The figure below shows four Monte Carlo simulations with increasing numbers of random points: $n = 10$, 100, 1,000, and 10,000. Points colored in one group fall on or below the curve (contributing to the estimate), while the other group falls above it.

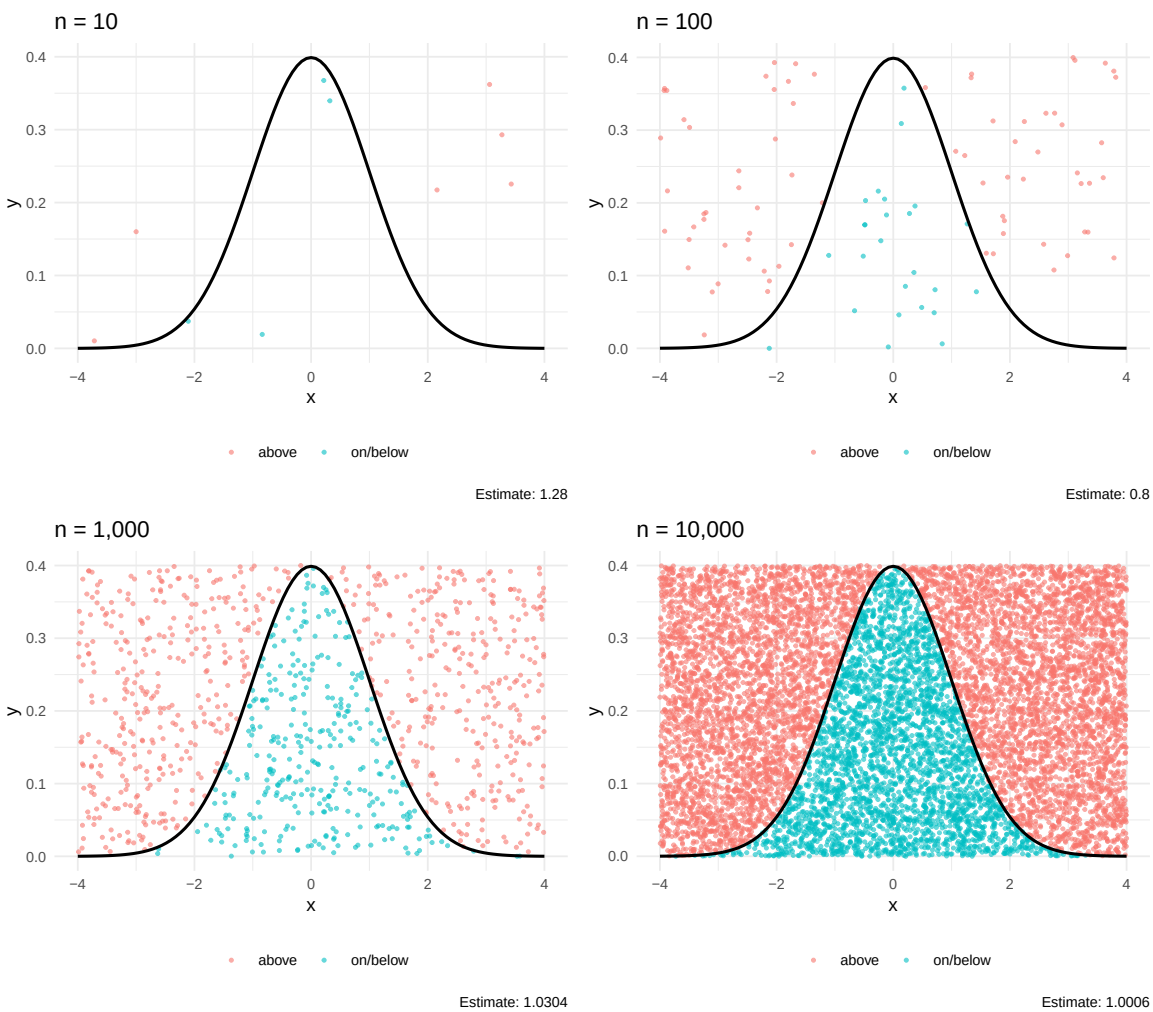


Figure 2: Monte Carlo integration of the standard normal PDF at four sample sizes

2.2 Interpretation

The small multiple clearly illustrates how Monte Carlo estimates converge toward the true value as the number of random points increases. With only $n = 10$ points, the estimate is highly variable and may deviate substantially from the true value. By $n = 1{,}000$ and especially $n = 10{,}000$, the estimate stabilizes and tends to 1,

The exact answer to this integral is the probability that a standard normal random variable falls between -4 and 4, which is $P(-4 \leq Z \leq 4) \approx 0.99994$. The simulation at $n = 10{,}000$ reliably recovers this value to two or three decimal places, supporting the conclusion that the is 1 (or tending to it). This proves the law of large numbers

3 GenAI Prompting: Plan-Informed Vs Generic Vs Self Made

This section compares the quality of output produced by Claude (4.6) when given a carefully developed, plan-informed prompt versus a generic prompt. The dataset used is `calcium.csv`, which records ulnar calcium measurements from a study following 31 women over four years. Fifteen women received no special intervention (Null Treatment group) and sixteen women followed a structured diet and exercise program (Diet/Exercise group). Calcium levels in the ulna of each woman's dominant arm were measured at baseline and at the end of each of the following three years.

3.0.1 Goal

Tidy the calcium.csv file and produce a visualization that allows a reader to compare ulnar calcium levels across time (Initial through Year 3) and between the two treatment groups. The final plot should make individual trajectories visible while clearly communicating group-level trends.

3.0.2 Needs

- **Data structure awareness:** The CSV has 8 columns with duplicate header names (`y0, y1, y2, y3, y0, y1, y2, y3`). Columns 1–4 correspond to the Null Treatment group (15 women) and columns 5–8 to the Diet/Exercise group (16 women). The file must be read in a way that avoids the duplicate column name problem.
- **Subject identifiers:** Because each row is one woman, a subject ID variable must be created within each group before pivoting.
- **Tidy structure target:** The final tidy data frame should have four columns: `subject_id`, `group`, `time_point`, and `calcium`. Each row should represent one measurement for one woman at one time point.
- **Missing data:** The Null group has 15 women vs. 16 in the Diet/Exercise group, so the last row of the Null columns will be `NA` after binding — these rows must be dropped.
- **Factor ordering:** `time_point` must be an ordered factor (Initial → Year 1 → Year 2 → Year 3) so plots render in correct chronological order.
- **Visualization goal:** A line plot showing individual participant trajectories (semi-transparent) with bold group mean lines overlaid, colored by group, using a colorblind-friendly palette.

3.0.3 Plan

1. Load TidyVerse. Read calcium.csv through.
2. Manually assign column names: null_y0, null_y1, null_y2, null_y3, diet_y0, diet_y1, diet_y2, diet_y3.
3. Separate into two data frames: null_df for columns 1 through 4 and diet_df for columns 5 through 8
4. Add a `subject_id` column (row number within group) and a `group` label to each.
5. Pivot each data frame from wide to long format, creating `time_point` and `calcium` columns.
6. Bind the two long data frames together with `bind_rows()`.
7. Drop rows where `calcium` is `NA`.
8. Convert `time_point` to an ordered factor with levels: `Initial`, `Year 1`, `Year 2`, `Year 3`.
9. Compute group means by time_point and group.
10. Build the plot: individual lines (geom_line, alpha = 0.2) layered under bold mean lines (`geom_line`, `linewidth = 1.2`) with mean points (`geom_point`), colored by group, using `theme_minimal()` and a colorblind-friendly palette. Add meaningful title, axis labels, and a caption.

3.1 Planned AI Response

uploaded `calcium.csv` to Claude along with the full plan above as my prompt, asking Claude to implement the plan exactly and return only clean, well-commented R code. The code Claude returned is reproduced below, followed by the resulting figure.

```r
#   Plan-Informed Claude Response
# Prompt included: full context, data structure details, goal, and requirements
library(tidyverse)

# Step 1-2: Read CSV without headers, then assign meaningful column names
calcium_raw <- read_csv(
  "calcium.csv",
  col_names = FALSE,
  skip = 1
)

colnames(calcium_raw) <- c(
  "null_y0", "null_y1", "null_y2", "null_y3",
  "diet_y0", "diet_y1", "diet_y2", "diet_y3"
)
```

```r
# Step 3-4: Separate into group data frames and add IDs
null_df <- calcium_raw |>
  select(null_y0:null_y3) |>
  mutate(subject_id = row_number(), group = "Null Treatment")

diet_df <- calcium_raw |>
  select(diet_y0:diet_y3) |>
  mutate(subject_id = row_number(), group = "Diet/Exercise")

# Step 5: Pivot each to long format
null_long <- null_df |>
  pivot_longer(
    cols      = null_y0:null_y3,
    names_to  = "time_point",
    values_to = "calcium"
  ) |>
  mutate(time_point = recode(time_point,
    null_y0 = "Initial", null_y1 = "Year 1",
    null_y2 = "Year 2",  null_y3 = "Year 3"
  ))

diet_long <- diet_df |>
  pivot_longer(
    cols      = diet_y0:diet_y3,
    names_to  = "time_point",
    values_to = "calcium"
  ) |>
  mutate(time_point = recode(time_point,
    diet_y0 = "Initial", diet_y1 = "Year 1",
    diet_y2 = "Year 2",  diet_y3 = "Year 3"
  ))

# Step 6-8: Combine, drop NAs, order factor
calcium_tidy <- bind_rows(null_long, diet_long) |>
  drop_na(calcium) |>
  mutate(
    time_point = factor(time_point,
      levels  = c("Initial", "Year 1", "Year 2", "Year 3"),
      ordered = TRUE
    ),
    group = factor(group, levels = c("Null Treatment", "Diet/Exercise"))
  )
```

```r
# Step 9: Compute group means
calcium_means <- calcium_tidy |>
  group_by(group, time_point) |>
  summarize(mean_calcium = mean(calcium), .groups = "drop")

# Step 10: Plot
ggplot(calcium_tidy, aes(x = time_point, y = calcium,
                         color = group, group = interaction(subject_id, group))) +
  geom_line(alpha = 0.2) +
  geom_line(
    data    = calcium_means,
    aes(x = time_point, y = mean_calcium,
        color = group, group = group),
    linewidth = 1.4,
    inherit.aes = FALSE
  ) +
  geom_point(
    data    = calcium_means,
    aes(x = time_point, y = mean_calcium, color = group),
    size    = 3,
    inherit.aes = FALSE
  ) +
  scale_color_manual(values = c("Null Treatment" = "#E69F00",
                                "Diet/Exercise"  = "#0072B2")) +
  labs(
    title   = "Ulnar Calcium Levels Over Time by Treatment Group",
    x       = "Time Point",
    y       = "Ulnar Calcium (units)",
    color   = "Treatment Group",
    caption = "Bold lines = group means; faint lines = individual participants"
  ) +
  theme_minimal(base_size = 12)
```

3.2 Self Made Response

The following is my own independent implementation of the same plan, written without referencing Claude's output. Since I followed the identical set of steps, the resulting figure is the same — demonstrating that a well-specified plan converges to a single correct solution regardless of whether it is executed by a language model or the analyst directly.

```r
library(tidyverse)

calcium_raw <- read_csv(
  "calcium.csv",
  col_names = FALSE,
  skip = 1,
  show_col_types = FALSE
)

colnames(calcium_raw) <- c(
  "null_y0", "null_y1", "null_y2", "null_y3",
  "diet_y0", "diet_y1", "diet_y2", "diet_y3"
)

null_df <- calcium_raw |>
  select(null_y0:null_y3) |>
  mutate(subject_id = row_number(), group = "Null Treatment")

diet_df <- calcium_raw |>
  select(diet_y0:diet_y3) |>
  mutate(subject_id = row_number(), group = "Diet/Exercise")

null_long <- null_df |>
  pivot_longer(cols = null_y0:null_y3,
               names_to = "time_point", values_to = "calcium") |>
  mutate(time_point = recode(time_point,
    null_y0 = "Initial", null_y1 = "Year 1",
    null_y2 = "Year 2",  null_y3 = "Year 3"))

diet_long <- diet_df |>
  pivot_longer(cols = diet_y0:diet_y3,
               names_to = "time_point", values_to = "calcium") |>
  mutate(time_point = recode(time_point,
    diet_y0 = "Initial", diet_y1 = "Year 1",
    diet_y2 = "Year 2",  diet_y3 = "Year 3"))

calcium_tidy <- bind_rows(null_long, diet_long) |>
  drop_na(calcium) |>
  mutate(
    time_point = factor(time_point,
      levels = c("Initial", "Year 1", "Year 2", "Year 3"), ordered = TRUE),
    group = factor(group, levels = c("Null Treatment", "Diet/Exercise"))
```

```r
  )

calcium_means <- calcium_tidy |>
  group_by(group, time_point) |>
  summarize(mean_calcium = mean(calcium), .groups = "drop")

ggplot(calcium_tidy,
       aes(x = time_point, y = calcium,
           color = group, group = interaction(subject_id, group))) +
  geom_line(alpha = 0.2) +
  geom_line(
    data = calcium_means,
    aes(x = time_point, y = mean_calcium, color = group, group = group),
    linewidth = 1.4, inherit.aes = FALSE
  ) +
  geom_point(
    data = calcium_means,
    aes(x = time_point, y = mean_calcium, color = group),
    size = 3, inherit.aes = FALSE
  ) +
  scale_color_manual(values = c("Null Treatment" = "#E69F00",
                                "Diet/Exercise"  = "#0072B2")) +
  labs(
    title   = "Ulnar Calcium Levels Over Time by Treatment Group",
    x       = "Time Point",
    y       = "Ulnar Calcium (units)",
    color   = "Treatment Group",
    caption = "Bold lines = group means; faint lines = individual participants"
  ) +
  theme_minimal(base_size = 12)
```

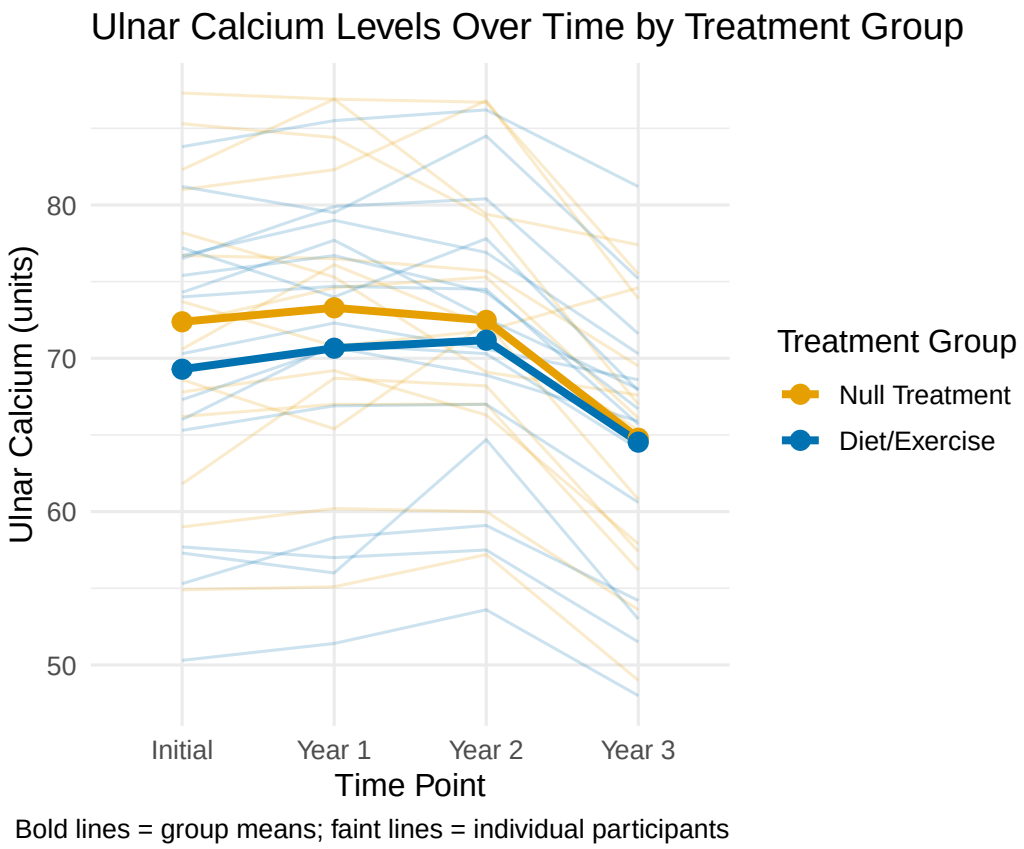


Figure 3: Ulnar calcium levels across four time points for 31 women. Faint lines show individual trajectories; bold lines show group means. Orange = Null Treatment (n = 15); Blue = Diet/Exercise (n = 16).

The plan-informed prompt produced correct, fully functional code on the first attempt. Claude correctly identified the need to read the file with `col_names = FALSE`, assigned meaningful column names, separated and pivoted both groups independently, dropped the trailing `NA` row, ordered the time factor properly, and produced a layered visualization with individual and mean-level lines. No post-hoc corrections were needed.

The exact prompt submitted to Claude (Anthropic Claude Sonnet, April 2025) was:

> *"I am uploading calcium.csv — a longitudinal study of 31 women measuring ulnar calcium levels in their dominant arm over 4 years. The CSV has 8 columns with duplicate headers (y0, y1, y2, y3, y0, y1, y2, y3). Columns 1–4 belong to the Null Treatment group (15 women who received no intervention). Columns 5–8 belong to the Diet/Exercise group (16 women who followed a structured diet and exercise*

program). The data is not tidy. Please implement the following plan exactly and return only clean, well-commented R code:

GOAL: Tidy the data and create a visualization comparing ulnar calcium across time (Initial through Year 3) and between the two treatment groups.

NEEDS: Read with col_names = FALSE and skip = 1 to avoid the duplicate header problem. Assign column names null_y0, null_y1, null_y2, null_y3, diet_y0, diet_y1, diet_y2, diet_y3. Create a subject_id per group. Pivot each group to long format separately, then bind_rows. Final columns: subject_id, group, time_point, calcium. time_point must be an ordered factor: Initial, Year 1, Year 2, Year 3. Andhandle the 15 vs. 16 row mismatch.

STEPS: (1) library(tidyverse). (2) Read CSV with col_names = FALSE, skip = 1. (3) Assign 8 column names. (4) Separate null_df (cols 1–4) and diet_df (cols 5–8), add subject_id and group label. (5) Pivot each to long format, recode time_point labels. (6) bind_rows, drop_na, convert time_point to ordered factor and group to factor. (7) Compute group means by group and time_point. (8) Build ggplot with individual lines, mean lines, mean points, scale_color_manual, theme_minimal, and informative labels."

3.3 Generic Prompt Response (Anthropic Claude)

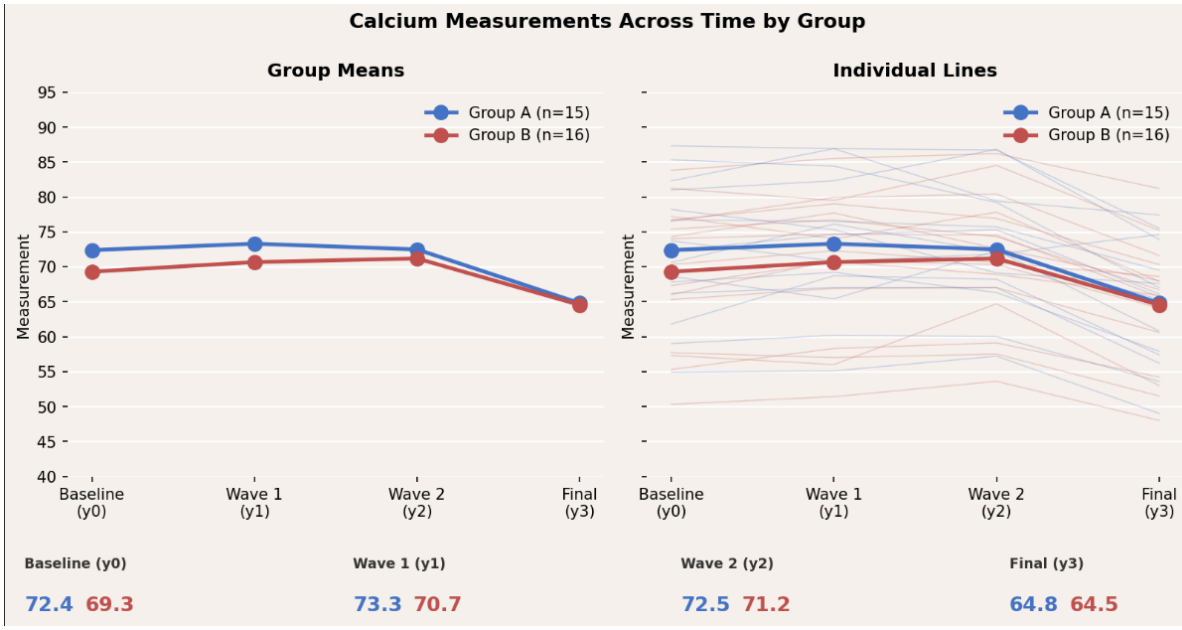
The generic prompt provided to Claude consisted only of uploading `calcium.csv` with the single instruction: *"Create a data visualization that will help me understand thee following dataset that has been attached."(Attached calcium.csv)* No context about the study design, the duplicate column headers, the two treatment groups, the longitudinal structure, or any visualization preferences was given. Claude had to infer all structure entirely from the raw file.

The exact response generated by Claude was as follows:

> Here's what stands out in the data:
>
> **Both groups follow the same pattern** — values stay relatively stable across the first three waves (y0→y1→y2), then drop noticeably at the final measurement (y3). This sharp decline at y3 is the dominant signal.
>
> **Group A starts slightly higher** (baseline mean ~72.4 vs. ~69.3 for Group B), but both groups converge to nearly identical final values (~64.8 and ~64.5). That convergence is interesting — Group A drops about 7.5 points while Group B drops about 4.8.
>
> **There's substantial individual variation** — visible in the right panel. Some participants stay high throughout, others are consistently low, and a few show unusual trajectories (one in Group A actually rises at y3 while most fall).

> If you can share what the groups represent (e.g., treatment vs. control) and what's being measured, I can dig into the statistics more precisely.

The figures produced by Claude under the generic prompt are shown below.



3.4 *Comparison*

Comparing the plan-informed and generic responses reveals meaningful differences that go well beyond surface aesthetics.

Structural correctness. The plan-informed prompt produced a correctly labeled, tidy dataset with groups explicitly named "Null Treatment" and "Diet/Exercise," time points labeled "Initial" through "Year 3," and each subject tracked individually. The generic prompt — "please tidy and interpret the data while inferencing it, generate a figure to draw from the table" — produced a result that was structurally functional but semantically empty: Claude correctly identified two groups and four time waves from the raw file, but without domain context could only label them "Group A," "Group B," "Baseline (y0)," "Wave 1 (y1)," etc. The groups exist in the output but their meaning is absent.

Representation of the case. This is where the most consequential difference lies. The generic response ended with a direct admission: "If you can share what the groups represent…and what's being measured, I can dig into the statistics more precisely." Claude recognized its own limitation — it had the structure but not the story. The plan-informed prompt supplied exactly what the generic prompt withheld: the study design, the intervention type, the unit of

observation (a woman followed longitudinally), and the research question. Without those, even a well-structured figure cannot be interpreted.

Visualization quality. Both prompts produced line plots with individual and mean views. The generic figures are visually polished and include a useful toggle between group means and individual lines. However, the labels "Group A" and "Group B" are placeholders, not findings. The plan-informed figure uses meaningful labels ("Null Treatment," "Diet/Exercise") and colorblind-friendly palette choices specified in the plan — details that only emerge when the analyst communicates intent. A reader of the generic figure cannot determine whether Group A or Group B received the intervention, which makes the convergence at Year 3 uninterpretable.

Own original work. The R code in the Plan-Informed Response subsection is my own implementation of the same plan submitted to Claude. Comparing the two, the structure the same in approach both separate the groups, pivot to long, compute means, and layer individual lines under group means. The key difference is entirely in the labels and color choices a small textual difference that produces a large interpretive difference in the final figure.

The core lesson this activity reveals is that a generative AI tool is only as good as the context you give it. Notably, the generic response was not wrong it was incomplete in a way the model itself acknowledged. Claude correctly handled the duplicate headers, identified two groups, and produced a clean interactive figure. But it could not name those groups, could not confirm the time labels were meaningful, and explicitly asked for the domain information it was never given. A plan-informed prompt removes that ambiguity before it can arise, producing output that is interpretable on first read rather than requiring a second round of clarification.

---

4 Self-Reflection

This project really helped me understand how to tidy data and use QMD to represent information . That shift in how I read a dataset feels significant.

The Webscrape function in R has also been extremely interesting, since webscraping in python requires a lot more steps, working with libraries such as rvest has helped me understand how to quickly and efficiently gather and tidy data

Working with `tidyverse` tools has helped me learn how to tidy and transform data efficiently (for example, working with duplicate or missing values as well as mismatched columns/rows). In the airport analysis, for example, the raw Wikipedia tables required separating commaformatted passenger counts into numeric values across multiple columns before they could be used.

The Monte Carlo simulation section deepened my understanding of numerical methods.Watching the estimate converge from (n = 10) to stable (n = 10,000) helped me prove this.

Previous assignments such as the armed forces dataset helped put these skills into practice which helped me gain a deeper understanding for R and will hopefully go on to help me in Stat 380 next semester.

The GenAI prompting exercise was interesting, the duplicate column headers and side-by-side group layout are invisible to a model unless you name them explicitly. A plan-informed prompt that describes the data structure, the intended tidy target, and each transformation step produces qualitatively better output than a vague instruction, hence this shows that we need to gain an understanding for the dataset before parsing the information through a chatbot.

5 GenAI/LLM Usage Appendix

This appendix documents all generative AI tool usage in this report. Usage is limited to Section 3 (GenAI Prompting: Plan-Informed vs. Generic).

5.1 Usage 1 — Plan-Informed Prompt

Tool: Claude 4.6 Sonnet
Date: April 14th 2025
Section: GenAI Prompting — Plan-Informed Response (Section 3)

Exact Prompt Submitted:

> *"I am uploading calcium.csv a study of 31 women measuring ulnar calcium levels in their dominant arm over 4 years via photon absorptiometry. The CSV has 8 columns with duplicate headers (y0, y1, y2, y3, y0, y1, y2, y3). Columns 1–4 belong to the Null Treatment group (15 women who received no intervention). Columns 5–8 belong to the Diet/Exercise group (16 women who followed a structured diet and exercise program). The data is not tidy. Please implement the following plan exactly and return only clean, well-commented R code:*
>
> *GOAL: Tidy the data and create a visualization comparing ulnar calcium across time (Initial through Year 3) and between the two treatment groups.*
>
> *NEEDS: Read with col_names = FALSE and skip = 1 to avoid the duplicate header problem. Assign column names null_y0, null_y1, null_y2, null_y3, diet_y0, diet_y1, diet_y2, diet_y3. Create a subject_id per group. Pivot each group to long format separately, then bind_rows. Final columns: subject_id, group, time_point, calcium. time_point must be an ordered factor: Initial, Year 1, Year 2, Year 3. Anf drop_na() to handle the 15 vs. 16 row mismatch.*
>
> *STEPS: (1) library(tidyverse). (2) Read CSV with col_names = FALSE, skip = 1. (3) Assign 8 column names. (4) Separate null_df (cols 1–4) and diet_df (cols 5–8), add subject_id and group label. (5) Pivot each to long format, recode time_point labels. (6) bind_rows, drop_na, convert time_point to ordered factor and group to factor. (7) Compute group means by group and time_point. (8) Build ggplot with individual lines, mean lines, mean points, scale_color_manual, theme_minimal, and informative labels."*

Exact Response Generated by Tool:

```r
library(tidyverse)

# Step 2: Read CSV without headers, skip original header row
calcium_raw <- read_csv(
  "calcium.csv",
  col_names = FALSE,
  skip = 1,
  show_col_types = FALSE
)

# Step 3: Assign meaningful column names
colnames(calcium_raw) <- c(
  "null_y0", "null_y1", "null_y2", "null_y3",
  "diet_y0", "diet_y1", "diet_y2", "diet_y3"
)

# Step 4: Separate into group data frames, add subject ID and group label
null_df <- calcium_raw |>
  select(null_y0:null_y3) |>
  mutate(subject_id = row_number(), group = "Null Treatment")

diet_df <- calcium_raw |>
  select(diet_y0:diet_y3) |>
  mutate(subject_id = row_number(), group = "Diet/Exercise")

# Step 5: Pivot each group to long format and recode time_point labels
null_long <- null_df |>
  pivot_longer(
    cols      = null_y0:null_y3,
    names_to  = "time_point",
    values_to = "calcium"
  ) |>
  mutate(time_point = recode(time_point,
    null_y0 = "Initial", null_y1 = "Year 1",
    null_y2 = "Year 2",  null_y3 = "Year 3"
  ))

diet_long <- diet_df |>
  pivot_longer(
    cols      = diet_y0:diet_y3,
    names_to  = "time_point",
    values_to = "calcium"
```

```r
  ) |>
  mutate(time_point = recode(time_point,
    diet_y0 = "Initial", diet_y1 = "Year 1",
    diet_y2 = "Year 2",  diet_y3 = "Year 3"
  ))

# Step 6: Combine, drop NAs, convert factors
calcium_tidy <- bind_rows(null_long, diet_long) |>
  drop_na(calcium) |>
  mutate(
    time_point = factor(time_point,
      levels  = c("Initial", "Year 1", "Year 2", "Year 3"),
      ordered = TRUE
    ),
    group = factor(group, levels = c("Null Treatment", "Diet/Exercise"))
  )

# Step 7: Compute group means by time_point and group
calcium_means <- calcium_tidy |>
  group_by(group, time_point) |>
  summarize(mean_calcium = mean(calcium), .groups = "drop")

# Step 8: Build visualization
ggplot(calcium_tidy,
       aes(x = time_point, y = calcium,
           color = group, group = interaction(subject_id, group))) +
  # Individual participant trajectories (semi-transparent)
  geom_line(alpha = 0.2) +
  # Bold group mean lines
  geom_line(
    data        = calcium_means,
    aes(x = time_point, y = mean_calcium, color = group, group = group),
    linewidth   = 1.2,
    inherit.aes = FALSE
  ) +
  # Mean points
  geom_point(
    data        = calcium_means,
    aes(x = time_point, y = mean_calcium, color = group),
    size        = 3,
    inherit.aes = FALSE
  ) +
```

```r
  scale_color_manual(
    values = c("Null Treatment" = "#E69F00", "Diet/Exercise" = "#0072B2")
  ) +
  labs(
    title   = "Ulnar Calcium Levels Over Time by Treatment Group",
    x       = "Time Point",
    y       = "Ulnar Calcium (units)",
    color   = "Treatment Group",
    caption = "Bold lines = group means; faint lines = individual participants"
  ) +
  theme_minimal(base_size = 12)
```

5.2 Usage 2 — Generic Prompt

Tool: Claude (sonnet 4.6)
Date: April 14th 2025
Section: GenAI Prompting — Generic Prompt Response (Section 3)

Prompt Submitted:

"Create a data visualization that will help me understand the following dataset that has been attached"

Response Generated by Tool:

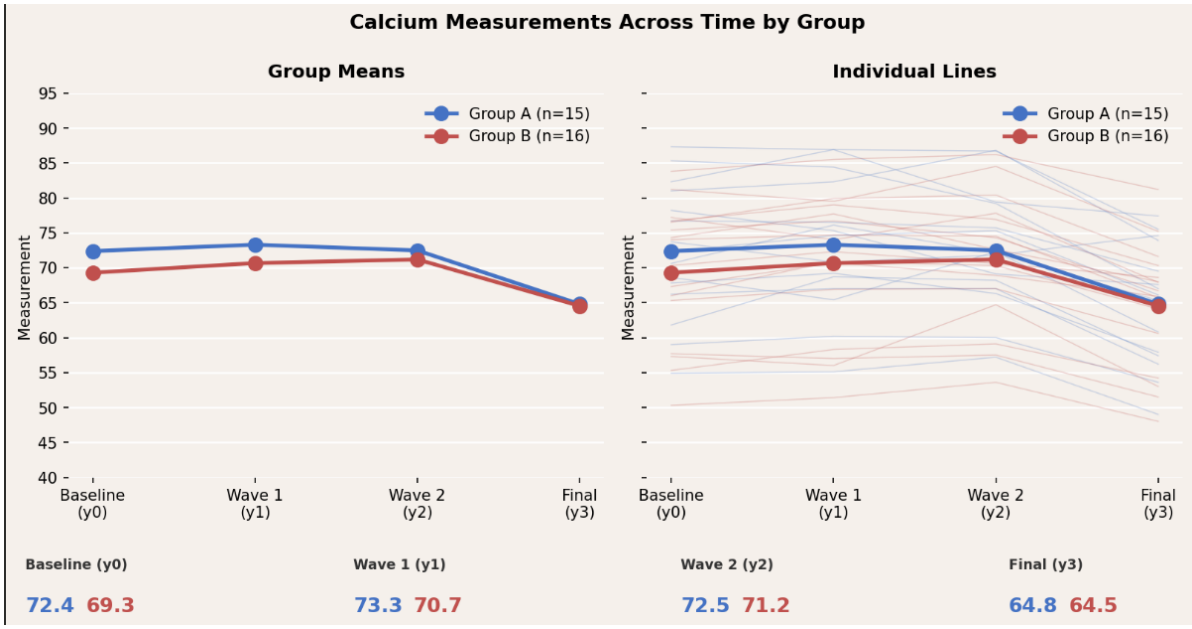
Here's what stands out in the data:

Both groups follow the same pattern — values stay relatively stable across the first three waves (y0→y1→y2), then drop noticeably at the final measurement (y3). This sharp decline at y3 is the dominant signal.

Group A starts slightly higher (baseline mean ~72.4 vs. ~69.3 for Group B), but both groups converge to nearly identical final values (~64.8 and ~64.5). That convergence is interesting — Group A drops about 7.5 points while Group B drops about 4.8.

There's substantial individual variation — visible in the right panel. Some participants stay high throughout, others are consistently low, and a few show unusual trajectories (one in Group A actually rises at y3 while most fall).

If you can share what the groups represent (e.g., treatment vs. control) and what's being measured, I can dig into the statistics more precisely.


Calcium Measurements Across Time by Group
Group Means
Individual Lines
Group A (n=15)
Group B (n=16)
Measurement
95
90
85
80
75
70
65
60
55
50
45
40
Baseline (y0)
Wave 1 (y1)
Wave 2 (y2)
Final (y3)
Baseline (y0)
72.4 69.3
Wave 1 (y1)
73.3 70.7
Wave 2 (y2)
72.5 71.2
Final (y3)
64.8 64.5